

Modelos en Sequelize

Primary Key, Foreign Key y cómo mapea la base de datos

1. ¿Qué es un modelo en Sequelize?

Un modelo es la representación en JavaScript de una tabla de la base de datos. Sequelize usa ese modelo para saber qué columnas tiene la tabla, qué tipo de datos guardan, y cómo se relacionan con otras tablas.

Tabla en PostgreSQL		Modelo en Sequelize
barras	<->	Barra
ventas	<->	Venta
bebidas	<->	Bebida
detalle_ventas	<->	DetalleVenta

2. Cómo se define un modelo (Primary Key incluida)

```
// src/models/barra.js
export default (sequelize, DataTypes) => {
  const Barra = sequelize.define('Barra', {

    // Columnas de la tabla
    numero_barra: {
      type: DataTypes.INTEGER,    // tipo de dato en la BD
      allowNull: false,          // NOT NULL
      unique: true,               // restriccion UNIQUE
    },
    sector: {
      type: DataTypes.STRING(100),
      allowNull: false,
    },
  }, {
    tableName: 'barras',        // nombre real de la tabla en PostgreSQL
    timestamps: false,          // no agrega createdAt / updatedAt
  });

  return Barra;
};
```

¿Dónde está la Primary Key?

Sequelize **agrega id automáticamente** como SERIAL PRIMARY KEY si no la declarás. Es equivalente a escribir esto (aunque no hace falta):

```
id: {
  type:      DataTypes.INTEGER,
  primaryKey: true,
  autoIncrement: true,
},
```

Si querés una PK con otro nombre o tipo, la declarás explícitamente y ponés `primaryKey: true`.

3. Cómo se exporta e importa el modelo

Archivo del modelo — exporta una función fábrica

```
// src/models/bebida.js
export default (sequelize, DataTypes) => {
  const Bebida = sequelize.define('Bebida', { /* columnas */ });
  return Bebida; // devuelve el modelo ya inicializado
};
```

La función recibe `sequelize` (la conexión) y `DataTypes` (los tipos) porque el modelo necesita estar atado a una conexión concreta.

Archivo index — centraliza todos los modelos

```
// src/models/index.js
import { Sequelize, DataTypes } from 'sequelize';
import definirBarra from './barra.js';
import definirBebida from './bebida.js';
import definirVenta from './venta.js';
import definirDetalle from './detalleventa.js';

// 1. Crear la conexión
const sequelize = new Sequelize(
  process.env.POSTGRES_DB,
  process.env.POSTGRES_USER,
  process.env.POSTGRES_PASSWORD,
  { host: process.env.POSTGRES_HOST, dialect: 'postgres' }
);

// 2. Inicializar cada modelo pasandole la conexión
const Barra = definirBarra(sequelize, DataTypes);
const Bebida = definirBebida(sequelize, DataTypes);
const Venta = definirVenta(sequelize, DataTypes);
const DetalleVenta = definirDetalle(sequelize, DataTypes);

// 3. Declarar las relaciones (Foreign Keys)
Venta.hasMany(DetalleVenta, { foreignKey: 'venta_id' });
DetalleVenta.belongsTo(Venta, { foreignKey: 'venta_id' });

// 4. Exportar todo junto
export { sequelize, Barra, Bebida, Venta, DetalleVenta };
```

Cualquier parte de la app importa desde acá:

```
import { Barra, Venta } from '../models/index.js';
```

4. Foreign Key — cómo relacionar dos modelos

Una Foreign Key (FK) es una columna en una tabla que apunta al id de otra tabla.

Ejemplo: DetalleVenta tiene FK a Venta, Bebida y Barra

ventas	bebidas	barras
■■■■■■■	■■■■■■■■■	■■■■■■■

Paso 1 — Declarar la columna FK en el modelo hijo

Paso 2 — Declarar la asociación en index.js

Las asociaciones le dicen a Sequelize *cómo* están relacionados los modelos, para que pueda hacer JOINS automáticos y agregar métodos de conveniencia.

Asociación	Cuándo usarla
------------	---------------

hasMany	Un registro padre tiene muchos hijos
belongsTo	Un registro hijo pertenece a un padre
hasOne	Un padre tiene exactamente un hijo
belongsToMany	Muchos a muchos (tabla intermedia)

5. Cómo mapea Sequelize cada tipo

DataTypes.*	Columna en PostgreSQL	Ejemplo de valor
INTEGER	INTEGER	42
BIGINT	BIGINT	9999999999
FLOAT	REAL	3.14
DECIMAL(10,2)	NUMERIC(10,2)	1234.56
STRING / STRING(n)	VARCHAR(255) / VARCHAR(n)	'Sector A'
TEXT	TEXT	'descripcion larga...'
BOOLEAN	BOOLEAN	true / false
DATE	TIMESTAMP WITH TIME ZONE	new Date()
DATEONLY	DATE	'2024-12-31'
UUID	UUID	'a1b2-...'

6. Resumen del flujo completo

1. Crear src/models/entidad.js
export default (sequelize, DataTypes) => sequelize.define(...)
2. Importar y llamar la funcion en src/models/index.js
const Entidad = definirEntidad(sequelize, DataTypes)
3. Si tiene FK -> declararla en la columna con references
y agregar la asociacion (hasMany / belongsTo) en index.js
4. Crear la migracion que genera la tabla en la BD
npx sequelize-cli migration:generate --name crear-entidad
5. Correr la migracion
npx sequelize-cli db:migrate
6. (Opcional) Crear el seeder con datos de prueba
npx sequelize-cli seed:generate --name datos-entidad
npx sequelize-cli db:seed:all